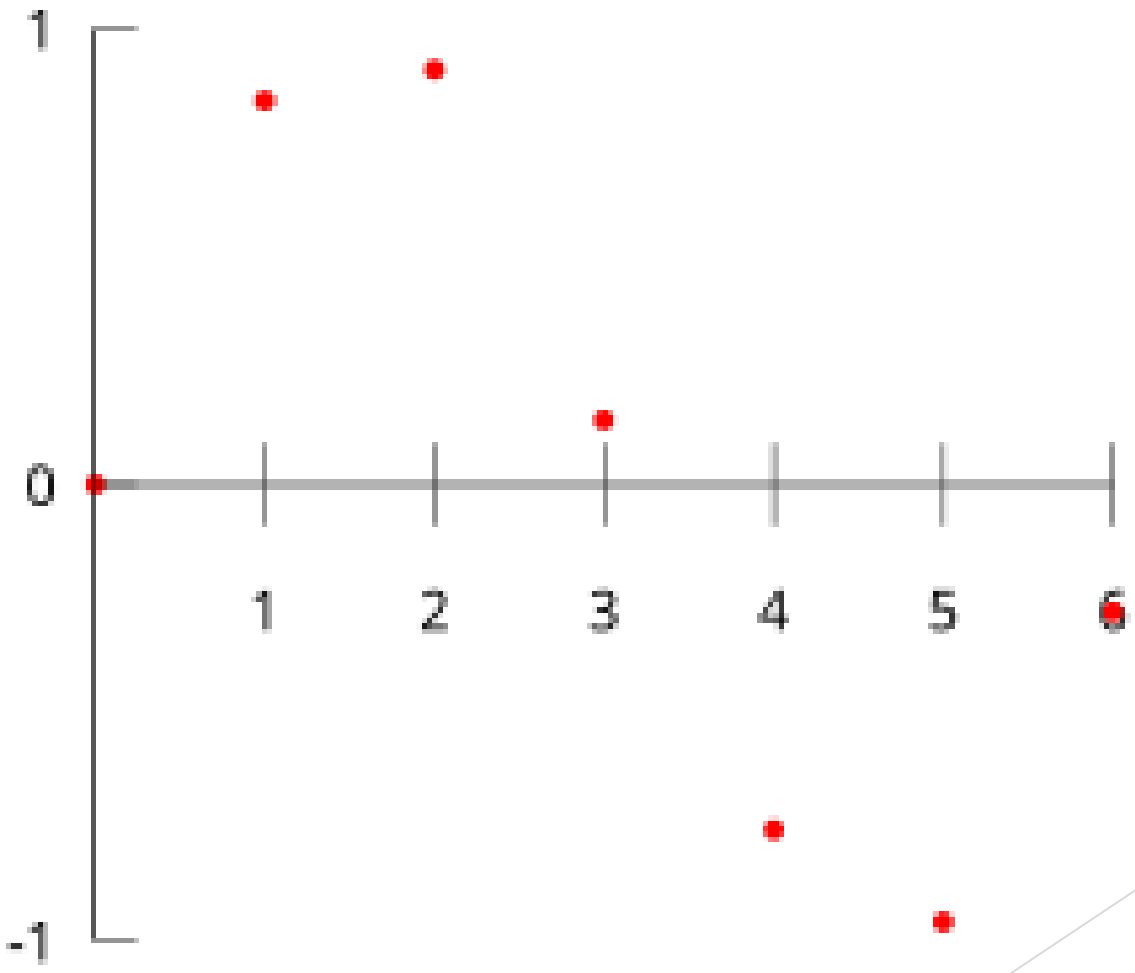
Interpolation

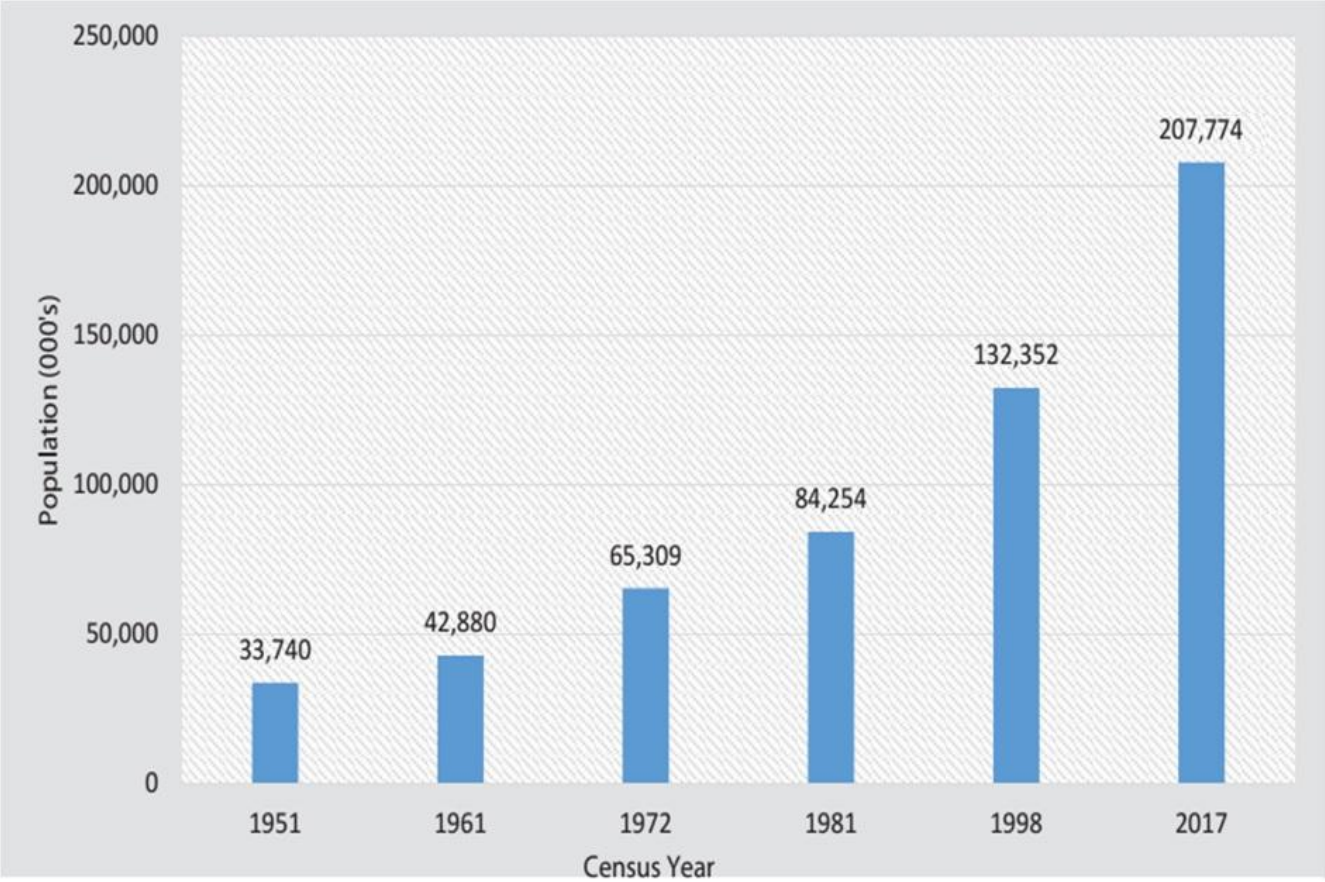
Introduction

Interpolation is a method of estimating new, unknown data points that lie between known, discrete data points

Interpolation



Interpolation



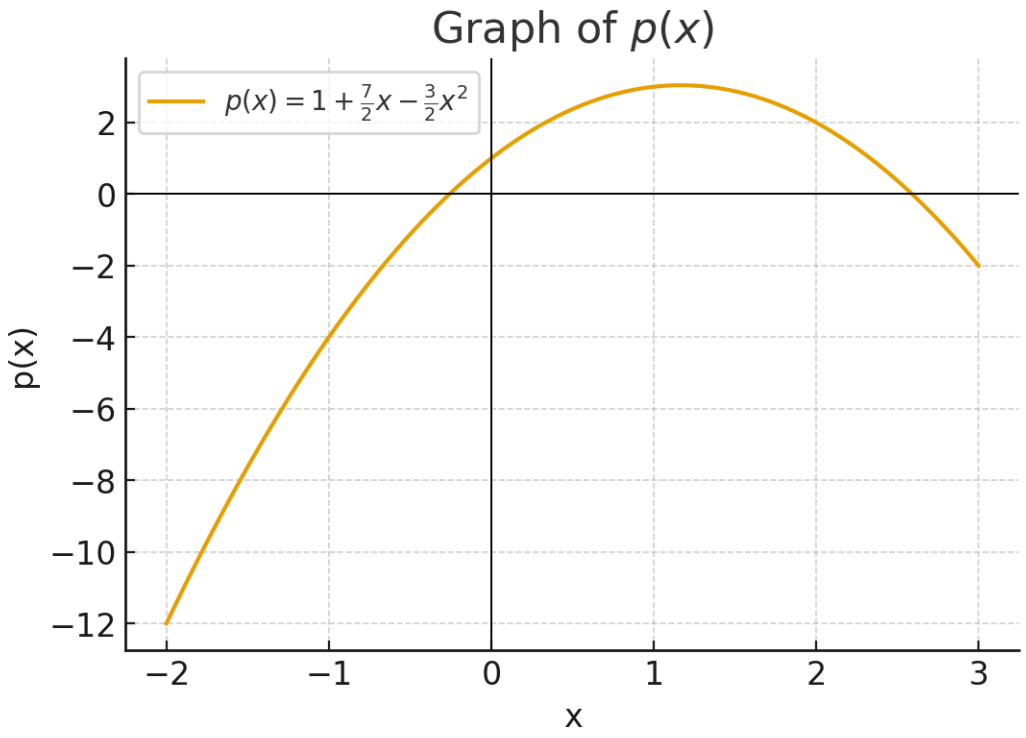
Introduction

Interpolation is the process of deriving a simple function from a set of discrete data points so that the function passes through all the given data points

Use **3 points**:

$$(0, 1), \quad (1, 3), \quad (2, 2).$$

We expect a quadratic $p(x) = a + bx + cx^2$.



Interpolation

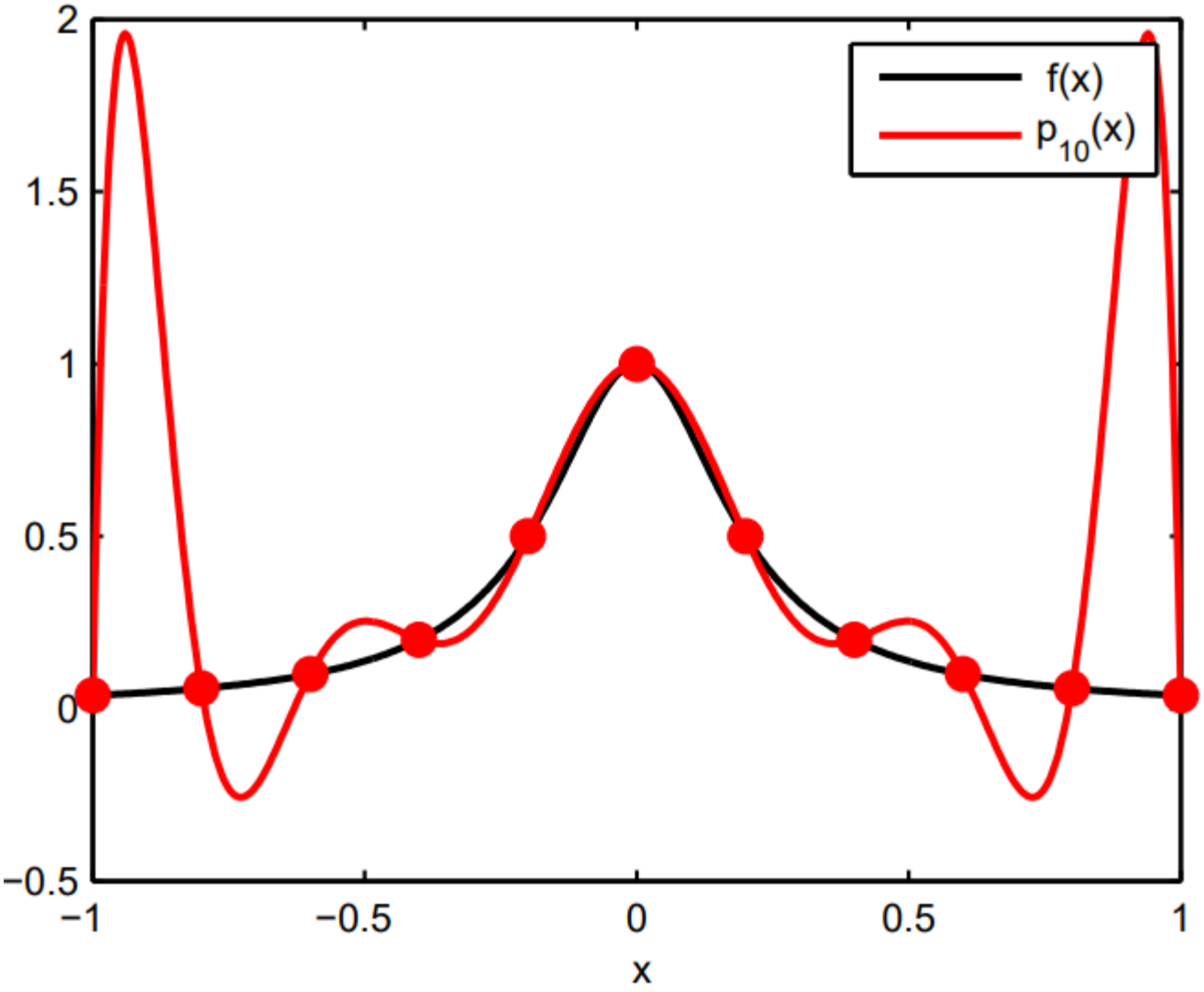
Given n+1 data points the direct method assumes the following polynomial:

$$y = f(x) = a_0 + a_1 x + a_2 x^2 + \ldots + a_n x^n$$

With n+1 values for x and the n+1 corresponding values for y we can solve for $a_0, a_1, \ldots, a_n$ by solving the n+1 simultaneous linear equations, which is known as the direct method.

Introduction

Interpolation is the problem of fitting a smooth curve through a given set of points, generally as the graph of a function.


f(x)
p10(x)
2
1.5
1
0.5
0
−0.5
−1
−0.5
0
0.5
1
x

Vandermonde matrix

Interpolation using the Vandermonde matrix

The most basic procedure to determine the coefficients $a_0, a_1, \ldots, a_n$ of a polynomial

$$P_n(x) = a_0 + a_1 x + a_2 x^2 + \cdots + a_n x^n$$

such that it interpolates the $n+1$ points

$$(x_0, y_0), (x_1, y_1), \ldots, (x_n, y_n)$$

is to write a linear system of equations as follows:

$$\begin{aligned}
P_n(x_0) = y_0 &\Rightarrow a_o + a_1 x_0 + a_2 x_0^2 + \cdots + a_{n-1} x_0^{n-1} + a_n x_0^n = y_0 \\
P_n(x_1) = y_1 &\Rightarrow a_o + a_1 x_1 + a_2 x_1^2 + \cdots + a_{n-1} x_1^{n-1} + a_n x_1^n = y_1 \\
\vdots &\Rightarrow \vdots \\
P_n(x_n) = y_n &\Rightarrow a_o + a_1 x_n + a_2 x_n^2 + \cdots + a_{n-1} x_n^{n-1} + a_n x_n^n = y_n
\end{aligned}$$

or, in matrix form:

$$\underbrace{\begin{bmatrix} 1 & x_0 & x_0^2 & \ldots & x_0^{n-1} & x_0^n \\ 1 & x_1 & x_1^2 & \ldots & x_1^{n-1} & x_1^n \\ \vdots & \vdots & \vdots & \vdots & \vdots & \\ 1 & x_{n-1} & x_{n-1}^2 & \ldots & x_{n-1}^{n-1} & x_{n-1}^n \\ 1 & x_n & x_n^2 & \ldots & x_n^{n-1} & x_n^n \end{bmatrix}}_{\mathbf{V}} \underbrace{\begin{bmatrix} a_0 \\ a_1 \\ \vdots \\ a_{n-1} \\ a_n \end{bmatrix}}_{\vec{a}} = \underbrace{\begin{bmatrix} y_0 \\ y_1 \\ \vdots \\ y_{n-1} \\ y_n \end{bmatrix}}_{\vec{b}}$$

Vandermonde matrix

Use the expression of p_N into the interpolating equations $y_j = p_N(x_j)$:

$$\sum_{n=0}^{N} a_n x_j^n = y_j, \qquad j = 0, \ldots, N.$$

In these $N+1$ equations indexed by j, the unknowns are the coefficients $a_0, \ldots, a_N$. We are in presence of a linear system

$$V\mathbf{a} = \mathbf{y}, \qquad \Leftrightarrow \qquad \sum_{n=0}^{N} V_{jn} a_n = y_j,$$

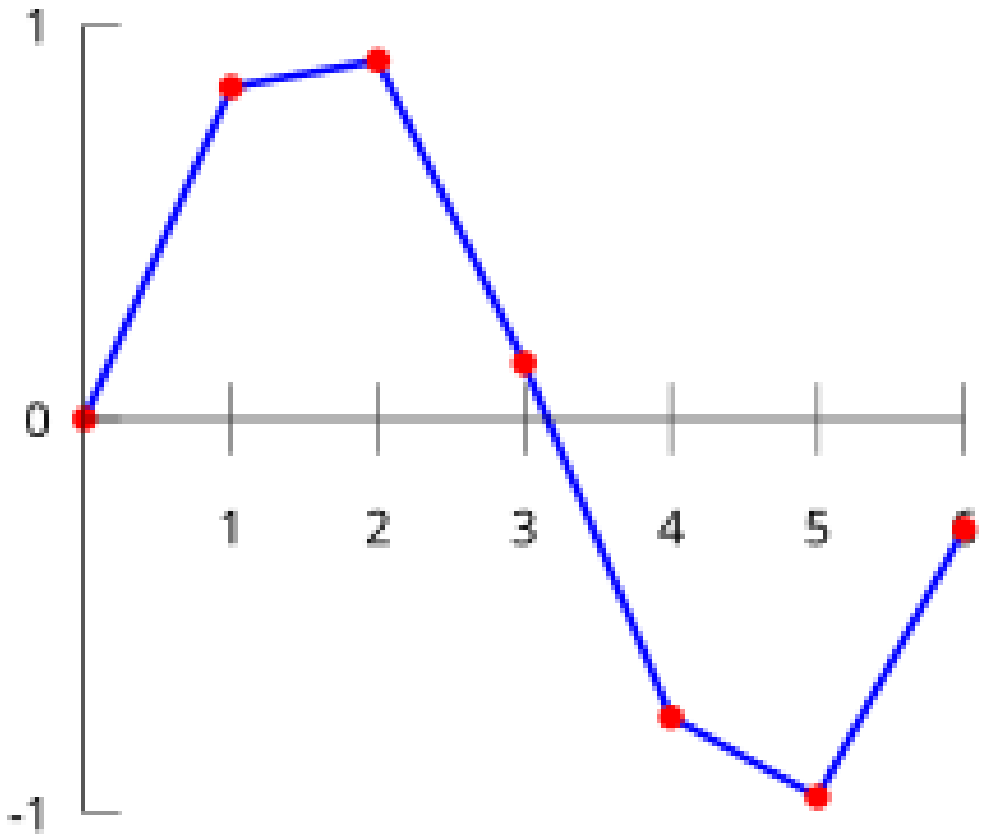
with V the so-called Vandermonde matrix, $V_{jn} = x_j^n$, i.e.,

$$V = \begin{pmatrix} 1 & x_0 & \cdots & x_0^N \\ 1 & x_1 & \cdots & x_1^N \\ \vdots & \vdots & & \vdots \\ 1 & x_N & \cdots & x_N^N \end{pmatrix}.$$

Types of Interpolation

- Linear interpolation
- Nearest Neighbor Interpolation
- Polynomial interpolation
- Spline interpolation

Linear interpolation



Linear interpolation

$$y = y_a + (y_b - y_a)\,\frac{x - x_a}{x_b - x_a} \text{ at the point } (x, y)$$

$$\frac{y - y_a}{y_b - y_a} = \frac{x - x_a}{x_b - x_a}$$

$$\frac{y - y_a}{x - x_a} = \frac{y_b - y_a}{x_b - x_a}$$

Slop of these two points will be the same as the slop of the polynomial

Nearest Neighbours Interpolation

Nearest neighbours interpolation is the simplest interpolation method.

$$(1, 2), (3, 5), (6, 1)$$

Polynomial interpolation

Given $N+1$ points $x_j \in \mathbb{R}$, $0 \leq j \leq N$, and sample values $y_j = f(x_j)$ of a function at these points, the polynomial interpolation problem consists in finding a polynomial $p_N(x)$ of degree N which reproduces those values:

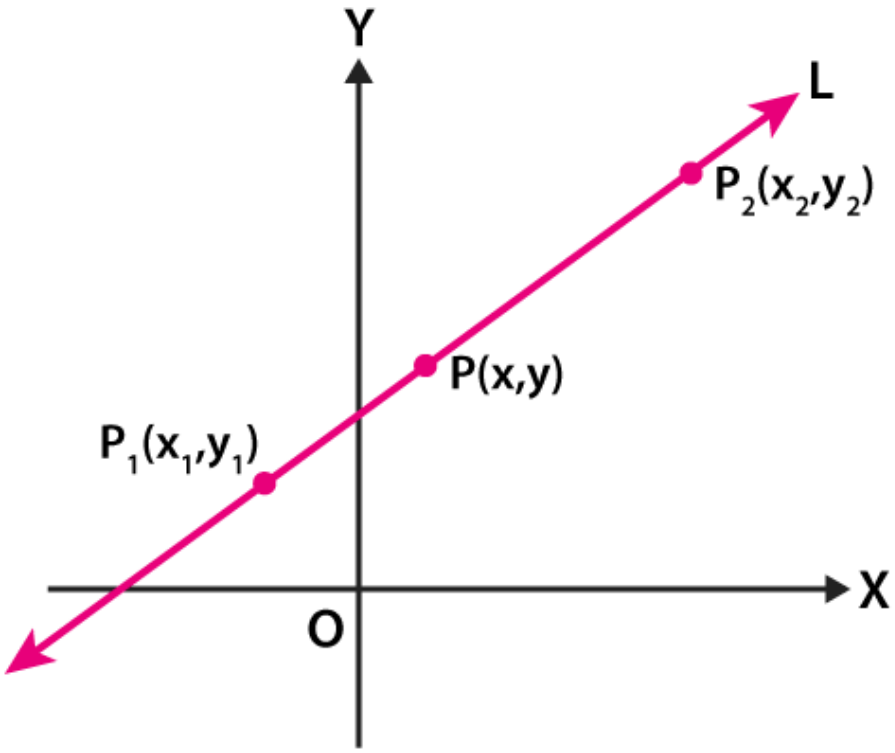
$$y_j = p_N(x_j), \qquad j = 0, \ldots, N.$$

Polynomial interpolation

Given a set of data, polynomial interpolation is a method of finding a polynomial function that fits a set of data points exactly. Though there are several methods for finding this polynomial

Quadratic Interpolation

Quadratic Interpolation



Quadratic Interpolation

$$y - y_1 = m(x - x_1) \quad \text{and} \quad m = \frac{y_1 - y_0}{x_1 - x_0}$$

$$y = \frac{y_1 - y_0}{x_1 - x_0}(x - x_1) + y_1$$

$$y = \frac{x - x_1}{x_1 - x_0}(y_1 - y_0) + y_1$$

$$y = \frac{x - x_1}{x_1 - x_0}y_1 - \frac{x - x_1}{x_1 - x_0}y_0 + y_1$$

$$y = \frac{x - x_1}{x_1 - x_0}y_1 - \frac{x - x_1}{x_1 - x_0}y_0 + \frac{x_1 - x_0}{x_1 - x_0}y_1$$

$$y = -\frac{x - x_1}{x_1 - x_0}y_0 + \frac{x - x_0}{x_1 - x_0}y_1$$

$$y = \frac{x - x_1}{x_0 - x_1}y_0 + \frac{x - x_0}{x_1 - x_0}y_1.$$

Quadratic Interpolation

$$y = \frac{x - x_1}{x_0 - x_1} y_0 + \frac{x - x_0}{x_1 - x_0} y_1.$$

For three data points $(x_0, y_0), (x_1, y_1), (x_2, y_2)$, we can derive a polynomial that behaves similarly:

$$y = \frac{(x - x_1)(x - x_2)}{(x_0 - x_1)(x_0 - x_2)} y_0 + \frac{(x - x_0)(x - x_2)}{(x_1 - x_0)(x_1 - x_2)} y_1 + \frac{(x - x_0)(x - x_1)}{(x_2 - x_0)(x_2 - x_1)} y_2$$

Lagrange Interpolation

Lagrange Polynomial (General Form)

Given $n+1$ data points $(x_0, y_0), (x_1, y_1), \ldots, (x_n, y_n)$ with $x_0 < x_1 < \cdots < x_n$, the Lagrange polynomial is the nth degree polynomial passing through each of these points and written as:

$$y = \frac{(x-x_1)(x-x_2)\cdots(x-x_n)}{(x_0-x_1)(x_0-x_2)\cdots(x_0-x_n)}y_0 + \frac{(x-x_0)(x-x_2)\cdots(x-x_n)}{(x_1-x_0)(x_1-x_2)\cdots(x_1-x_n)}y_1 + \cdots + \frac{(x-x_0)(x-x_1)\cdots(x-x_{n-1})}{(x_n-x_0)(x_n-x_1)\cdots(x_n-x_{n-1})}y_n$$

Equivalently, we can use product notation:

$$y = \sum_{i=0}^{n} y_i \left(\prod_{\substack{j=0 \\ j \neq i}}^{n} \frac{x-x_j}{x_i-x_j} \right)$$

```
import numpy as np
import matplotlib.pyplot as plot

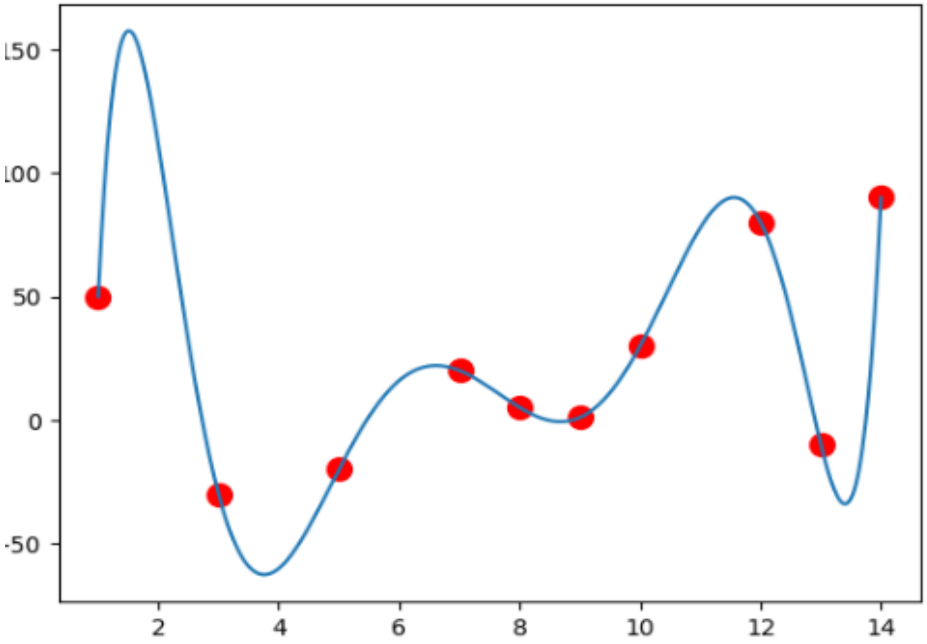
x=[1,3,5,7,8,9,10,12,13, 14]
y=[50,-30,-20,20,5,1,30,80,-10, 90]

n=len(x)-1
prange = np.linspace(min(x),max(x),500)

plot.plot(x,y,marker='o', color='r', ls='', markersize=10)

def f(o):
  sum = 0
  for i in range(n+1):
    prod = y[i]
    for j in range(n+1):
      if i!= j:
        prod=prod*(o-x[j])/(x[i]-x[j])
    sum = sum + prod
  return sum

plot.plot(prange,f(prange))
plot.show()
```



Home Task: Lagrange Interpolation (5-Marks)

- Implement Lagrange interpolation manually in Python.
- Identify and use the SciPy library function that provides Lagrange interpolation.
- Compare both on a 3rd-order any polynomial using 5 data points.
- Plot results and discuss differences (if any).

Topics

- Interpolation
- Linear interpolation
- Nearest Neighbor Interpolation
- Polynomial interpolation
  - Lagrange Interpolation
  - Newton's Interpolation

Newton's Divided Difference Interpolation

Newton's divided difference interpolation formula is an interpolation technique used when the interval difference is not same for all sequence of values.

Newton's Divided Difference Interpolation

Construct a polynomial that interpolates data points $(x_0, f(x_0)), (x_1, f(x_1)), \ldots, (x_n, f(x_n))$.

Newton's polynomial uses a form:

$$P_n(x) = a_0 + a_1(x - x_0) + a_2(x - x_0)(x - x_1) + \cdots + a_n(x - x_0)(x - x_1)\cdots(x - x_{n-1})$$

Newton's Divided Difference Interpolation

$$f(x) \approx a_0 + a_1 x + a_2 x^2 + \cdots + a_n x^n$$

$$P_n(x) = a_0 + a_1 x + a_2 x^2 + \cdots + a_n x^n$$

$$P_n(x) = a_0 + a_1(x - x_0) + a_2(x - x_0)(x - x_1) + \cdots + a_n(x - x_0)(x - x_1)\cdots(x - x_{n-1})$$

At $x = x_0$: $a_0 = f(x_0)$.

At $x = x_1$:

$$a_1 = \frac{f(x_1) - f(x_0)}{x_1 - x_0}$$

At $x = x_2$:

$$a_2 = \frac{\big(f(x_2) - f(x_1)\big)/(x_2 - x_1) - \big(f(x_1) - f(x_0)\big)/(x_1 - x_0)}{x_2 - x_0}$$

Newton's Divided Difference Interpolation

- Generally, Difference

$$f[x_i] = f(x_i),$$

$$f[x_i, x_{i+1}] = \frac{f(x_{i+1}) - f(x_i)}{x_{i+1} - x_i},$$

$$f[x_i, \ldots, x_{i+k}] = \frac{f[x_{i+1}, \ldots, x_{i+k}] - f[x_i, \ldots, x_{i+k-1}]}{x_{i+k} - x_i}$$

Newton's Divided Difference Interpolation

- Newton’s Formula

$$P_n(x) = f[x_0] + (x - x_0)f[x_0, x_1] + (x - x_0)(x - x_1)f[x_0, x_1, x_2] + \cdots$$

$$\cdot\cdot\,(x - x_{n-1})f[x_0, \ldots, x_n]$$

Newton's Divided Difference Interpolation

- Example

Compute $f(9.2)$ from the values shown in the following table.

x_j	$f_j = f(x_j)$
8.0	2.079442
9.0	2.197225
9.5	2.251292
11.0	2.397895

Example

To find $f[x_0,x_1]$, $f[x_0,x_1,x_2]$, $f[x_0,\ldots,x_n]$ in Newton's interpolation formula, we need to construct a **difference table** as follows:

x_j	$f_j = f(x_j)$	$f[x_j, x_{j+1}]$	$f[x_j, x_{j+1}, x_{j+2}]$	$f[x_j, \cdots, x_{j+3}]$
8.0	2.079442			
		0.117783		
9.0	2.197225		−0.006433	
		0.108134		0.000411
9.5	2.251292		−0.005200	
		0.097735		
11.0	2.397895			

The values in the table are calculated as follows:

First column:

$$f[x_j,x_{j+1}]=\frac{2.197225-2.079442}{9-8}=0.117783$$

Example

$$f[x_j, x_{j+1}] = \frac{2.251292 - 2.197225}{9.5 - 9} = 0.108134$$

$$f[x_j, x_{j+1}] = \frac{2.397895 - 2.251292}{11 - 9.5} = 0.097735$$

Second column:

$$f[x_j, x_{j+1}, x_{j+2}] = \frac{0.108134 - 0.117783}{9.5 - 8} = -0.006433$$

$$f[x_j, x_{j+1}, x_{j+2}] = \frac{0.097735 - 0.108134}{11 - 9} = -0.0052$$

Third column:

$$f[x_j, \ldots, x_{j+3}] = \frac{-0.0052 + 0.006433}{11 - 8} = 0.000411$$

Example

The values needed in Newton's interpolation formula are circled:

$$f(x) \approx p_3(x) = 2.079442 + 0.117783(x - 8.0) - 0.006433(x - 8.0)(x - 9.0) + 0.000411(x - 8.0)(x - 9.0)(x - 9.5).$$

At $x = 9.2$,

$$f(9.2) \approx 2.079442 + 0.141340 - 0.001544 - 0.000030 = 2.219208.$$

Newton's Divided Difference Interpolation

- Previously, we studied **Newton's Divided Difference Formula**, where data points were **not equally spaced**.
- In practice, however, many datasets are recorded at **equal intervals** (e.g., time series, experimental data, tabular values).
- For such cases, we use:
  - **Newton's Forward Difference Formula (Δ)** – interpolation near the **beginning** of the table.
  - **Newton's Backward Difference Formula (∇)** – interpolation near the **end** of the table.

Δ (Delta) → Forward difference symbol

∇ (Nabla) → Backward difference symbol

Newton's Forward & Backward Difference

When data points are **equally spaced**, Newton's **forward** and **backward difference interpolation formulas** provide efficient ways to approximate function values.

- **Forward Difference Formula** → Best used when the interpolation point x is **near the beginning** of the data table.
- **Backward Difference Formula** → Best used when the interpolation point x is **near the end** of the data table.

For equally spaced points:

$$x_0,\ x_1 = x_0 + h,\ x_2 = x_0 + 2h, \ldots$$

where h is the spacing.

Newton's Forward & Backward Difference

The **forward difference operator** Δ is defined as:

$$\Delta f(x_j) = f(x_{j+1}) - f(x_j)$$

Higher-order forward differences:

$$\Delta^2 f(x_j) = \Delta f(x_{j+1}) - \Delta f(x_j), \quad \Delta^3 f(x_j) = \Delta^2 f(x_{j+1}) - \Delta^2 f(x_j), \ \ldots$$

Newton's Forward & Backward Difference

In this case, Newton's Forward Difference Formula can be used to carry out an interpolation process. For a function $f(x)$, this formula can be expressed as:

$$f(x) = f_0 + r\,\Delta f_0 + \frac{r(r-1)}{2!}\Delta^2 f_0 + \ldots.. + \frac{r(r-1)\ldots\ldots(r-n-1)}{n!}\Delta^n f_0$$

$$r = \frac{x - x_j}{h}$$

Newton's Forward & Backward Difference

Example: Compute cosh (0.56) from Newton's forward difference formula and the four values in the following table.

j	x_j	$f_j = \cosh x_j$
0	0.5	1.127626
1	0.6	1.185465
2	0.7	1.255169
3	0.8	1.337435

Newton's Forward & Backward Difference

We compute the forward differences as shown in the table below.

j	x_j	$f_j = \cosh x_j$	Δf_j	$\Delta^2 f_j$	$\Delta^3 f_j$
0	0.5	1.127626			
			0.057839		
1	0.6	1.185465		0.011865	
			0.069704		0.000697
2	0.7	1.255169		0.012562	
			0.082266		
3	0.8	1.337435			

Newton's Forward & Backward Difference

The values we need are circled.

In the general formula, we have $r = (0.56\text{-}0.5)/0.1 = 0.6$. So that the general formula gives:

$$\cosh 0.56 \approx 1.127626 + 0.6 \cdot 0.057839 + \frac{0.6(-0.4)}{2} \cdot 0.011865 + \frac{0.6(-0.4)(-1.4)}{6} \cdot 0.000697$$

$$= 1.127626 + 0.034703 - 0.001424 + 0.000039$$

$$= 1.160944.$$

Newton's Forward & Backward Difference

The **backward difference operator** ∇ is defined as:

$$\nabla f(x_j) = f(x_j) - f(x_{j-1})$$

Higher-order backward differences:

$$\nabla^2 f(x_j) = \nabla f(x_j) - \nabla f(x_{j-1}), \quad \nabla^3 f(x_j) = \nabla^2 f(x_j) - \nabla^2 f(x_{j-1}), \ \ldots$$

Newton's Forward & Backward Difference

Therefore, ***the first backward difference of f at x_j is defined by :***

$$\nabla f_j = f_j - f_{j-1},$$

the *second backward difference* of f at x_j by

$$\nabla^2 f_j = \nabla f_j - \nabla f_{j-1},$$

and, continuing in this way, the ***k*th backward difference** of f at x_j by

$$\nabla^k f_j = \nabla^{k-1} f_j - \nabla^{k-1} f_{j-1} \qquad (k = 1, 2, \cdots).$$

A formula similar to the forward formula but involving backward differences is Newton's backward difference interpolation formula

$$f(x) = f_0 + r\,\nabla f_0 + \frac{r(r-1)}{2!}\nabla^2 f_0 + \ldots\ldots + \frac{r(r-1)\ldots\ldots(r-n-1)}{n!}\nabla^n f_0$$

Newton's Forward & Backward Difference

Try to solve example on the paper

Hermite Interpolation

- Ordinary interpolation (like Lagrange or Newton) uses only function values $f(x_i)$.
- Sometimes, we also know **derivatives** $f'(x_i)$.
- **Hermite interpolation** uses both, giving a more accurate polynomial.

Hermite Interpolation

For points $x_0, x_1, \ldots, x_n$ with known values $f(x_i)$ and derivatives $f'(x_i)$,
we can construct a polynomial of degree $\leq 2n + 1$ such that:

$$P(x_i) = f(x_i), \quad P'(x_i) = f'(x_i) \quad \text{for } i = 0, 1, \ldots, n$$

This polynomial is called the **Hermite interpolant**.